

**Laxmi Charitable Trust's**  
**Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce**  
**Department of Information Technology (B.Sc.I.T Semester V)**  
**Applied Business Analytics**

**Practical – 10**

|                              |                                   |
|------------------------------|-----------------------------------|
| Roll No.: T017               | Name: Radhika Mali                |
| Class: TYIT                  | Batch: 1                          |
| Date of Assignment: 03/10/26 | Date/Time of Submission: 03/10/26 |

**Part A: Regression Analysis**

**Dataset:** sales\_data.csv

**1. Load the sales\_data.csv dataset.**

**Code:**

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error
```

```
# Load the dataset
data = pd.read_csv("sales_data.csv")
```

**2. Display the first five records.**

**Code:**

```
# Display the first five records
print("First Five Records:")
print(data.head())
```

**Output:**

| First Five Records: |                   |                      |                          |      |
|---------------------|-------------------|----------------------|--------------------------|------|
| Unnamed: 0          | TV Ad Budget (\$) | Radio Ad Budget (\$) | Newspaper Ad Budget (\$) | \    |
| 0                   | 1                 | 230.1                | 37.8                     | 69.2 |
| 1                   | 2                 | 44.5                 | 39.3                     | 45.1 |
| 2                   | 3                 | 17.2                 | 45.9                     | 69.3 |
| 3                   | 4                 | 151.5                | 41.3                     | 58.5 |
| 4                   | 5                 | 180.8                | 10.8                     | 58.4 |

| Sales (\$) |      |
|------------|------|
| 0          | 22.1 |
| 1          | 10.4 |
| 2          | 9.3  |
| 3          | 18.5 |
| 4          | 12.9 |

**3. Identify the independent and dependent variables.**

**Code:**

```
#Identify independent and dependent variables
print("Independent Variable: TV Ad Budget ($)")
print("Dependent Variable: Sales ($)")
```

**Output:**

```
Independent Variable: TV Ad Budget ($)
Dependent Variable: Sales ($)
```

**4. Select advertising as the independent variable and sales as the dependent variable.**

**Code:**

```
#Select independent and dependent variables
X = data[["TV Ad Budget ($)"]]
y = data["Sales ($)"]
```

```
print(X.head())
```

```
print(y.head())
```

**Output:**

| TV Ad Budget (\$) |       |
|-------------------|-------|
| 0                 | 230.1 |
| 1                 | 44.5  |
| 2                 | 17.2  |
| 3                 | 151.5 |
| 4                 | 180.8 |

|   |      |
|---|------|
| 0 | 22.1 |
| 1 | 10.4 |
| 2 | 9.3  |
| 3 | 18.5 |
| 4 | 12.9 |

Name: Sales (\$), dtype: float64

**5. Check the data for missing values.**

**Code:**

```
#check missing values
print(data.isnull().sum())
```

**Output:**

```
Unnamed: 0      0
TV Ad Budget ($)      0
Radio Ad Budget ($)      0
Newspaper Ad Budget ($)      0
Sales ($)      0
dtype: int64
```

**6. Create a scatter plot of advertising expenditure and sales.**

**Code:**

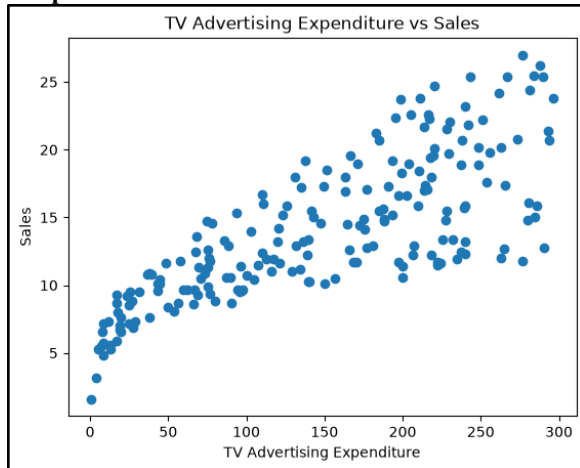
```
#Create scatter plot
import matplotlib.pyplot as plt
```

```
plt.scatter(data["TV Ad Budget ($)"], data["Sales ($)"])
```

```
plt.xlabel("TV Advertising Expenditure")
plt.ylabel("Sales")
plt.title("TV Advertising Expenditure vs Sales")
```

```
plt.show()
```

**Output:**



### 7. Develop a simple linear regression model.

**Code:**

```
#Develop Simple Linear Regression Model
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

```
print("Linear Regression Model Created")
```

**Output:**

```
Linear Regression Model Created
```

### 8. Display the regression equation.

**Code:**

```
#Display Regression Equation
b0 = model.intercept_
b1 = model.coef_[0]
print("Regression Equation:")
print(f"Sales = {b0:.2f} + {b1:.2f} × Advertising")
```

**Output:**

```
Regression Equation:
Sales = 7.12 + 0.05 × Advertising
```

### 9. Use the model to predict sales for a given advertising expenditure.

**Code:**

```
#Predict Sales
advertising_value = 100
predicted_sales =
model.predict([[advertising_value]])
print("Advertising Expenditure:", advertising_value)
print("Predicted Sales:", predicted_sales[0])
```

**Output:**

```
Advertising Expenditure: 100
Predicted Sales: 11.772611801137288
```

### 10. Calculate the R2 score of the model.

**Code:**

```
#Calculate R2 Score
from sklearn.metrics import r2_score
y_pred = model.predict(X_test)
r2 = r2_score(y_test, y_pred)
print("R2 Score:", r2)
```

**Output:**

```
R2 Score: 0.6766954295627076
```

### 11. Calculate the Mean Squared Error (MSE).

**Code:**

```
#Calculate MSE
from sklearn.metrics import mean_squared_error
mse = mean_squared_error(y_test, y_pred)
print("Mean Squared Error:", mse)
```

**Output:**

```
Mean Squared Error: 10.204654118800956
```

### 12. Compare the actual sales with the predicted sales.

**Code:**

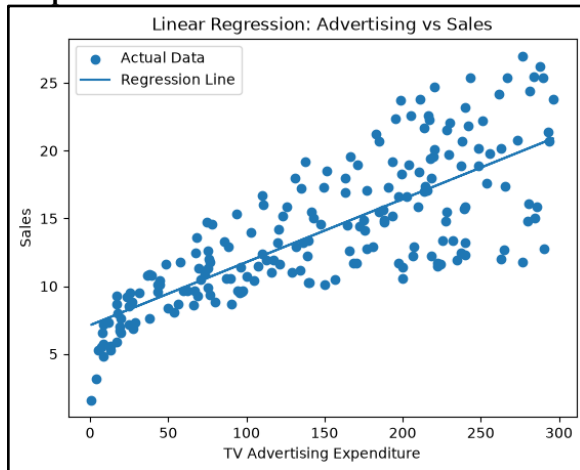
```
#Compare Actual and Predicted Sales
comparison = pd.DataFrame({
    "Actual Sales": y_test.values,
    "Predicted Sales": y_pred
})
print(comparison)
```

**Output:**

|   | Actual Sales | Predicted Sales |
|---|--------------|-----------------|
| 0 | 16.9         | 14.717944       |
| 1 | 22.4         | 16.211548       |
| 2 | 21.4         | 20.748197       |
| 3 | 7.3          | 7.664036        |
| 4 | 24.7         | 17.370139       |
| 5 | 12.6         | 10.614021       |
| 6 | 22.3         | 17.207285       |

**13. Plot the regression line with the actual data points.****Code:**

```
#Plot Regression Line
plt.scatter(X, y, label="Actual Data")
plt.plot(X, model.predict(X), label="Regression Line")
plt.xlabel("TV Advertising Expenditure")
plt.ylabel("Sales")
plt.title("Linear Regression: Advertising vs Sales")
plt.legend()
plt.show()
```

**Output:****14. Interpret the regression results.****Code:**

```
#Interpret Regression Results
print("Interpretation:")
print(f"For every 1 unit increase in advertising, sales increase by approximately {b1:.2f} units.")
print(f"R2 Score = {r2:.4f}")
```

**Output:**

```
Interpretation:
For every 1 unit increase in advertising, sales increase by approximately 0.05 units.
R2 Score = 0.6767
```

**15. Write a conclusion about how advertising expenditure is related to sales.****A simple regression model is:  $Sales = b_0 + b_1(Advertising)$ :**

There is a positive relationship between TV advertising expenditure and sales.

As the advertising expenditure increases, sales generally increase.

The regression model can be used to predict sales based on advertising expenditure.

**Part B: Time Series Analysis****Steps:****16. Load monthly sales data.****Code:**

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.holtwinters import
ExponentialSmoothing
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_absolute_error
data = pd.read_csv("monthly_sales.csv")
print("Top 5 rows:")
print(data.head())
print("\nColumn names:")
print(data.columns)
```

**Output:**

```
Top 5 rows:
   month  sales
0  Jan-22   120
1  Feb-22   135
2  Mar-22   128
3  Apr-22   145
4  May-22   150

Column names:
Index(['month', 'sales'], dtype='str')
```

**17. Convert the date column into date format.****Code:**

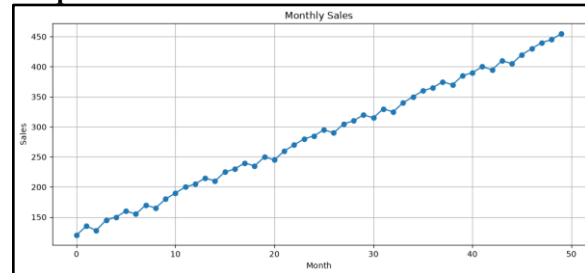
```
data['month'] = pd.to_datetime(data['month'],
format='%b-%Y')
data = data.set_index('month')
print(data.head())
print(data.index)
```

**Output:**

| month                                                                                                                                                                                                                    | sales |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| 1-01-22                                                                                                                                                                                                                  | 120   |
| 1-02-22                                                                                                                                                                                                                  | 135   |
| 1-03-22                                                                                                                                                                                                                  | 128   |
| 1-04-22                                                                                                                                                                                                                  | 145   |
| 1-05-22                                                                                                                                                                                                                  | 150   |
| DatetimeIndex(['1-01-22', '1-02-22', '1-03-22', '1-04-22', '1-05-22',<br>'1-06-22', '1-07-22', '1-08-22', '1-09-22', '1-10-22',<br>'1-11-22', '1-12-22', '1-01-23', '1-02-23', '1-03-23'],<br>dtype='object', freq='MS') |       |

**18. Plot monthly sales using a line chart.****Code:**

```
plt.figure(figsize=(12, 5))
plt.plot(data.index, data['sales'], marker='o')
plt.title("Monthly Sales")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.grid(True)
plt.show()
```

**Output:****19. Observe the trend.****Code:**

```
print("The sales show an overall increasing trend  
over time.")
```

**Output:**

```
The sales show an overall increasing trend over time.
```

**20. Apply Exponential Smoothing.****Code:**

```
es_model = ExponentialSmoothing(
    data['sales'],
    trend='add',
    seasonal=None
)
es_fit = es_model.fit()
print("Exponential Smoothing applied successfully.")
```

**Output:**

```
Exponential Smoothing applied successfully.
```

**21. Forecast the next 3 months.****Code:**

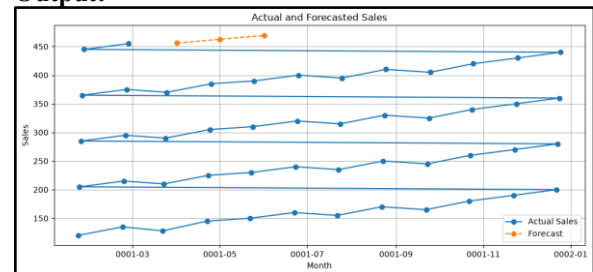
```
es_forecast = es_fit.forecast(3)
print("Exponential Smoothing Forecast:")
print(es_forecast)
```

**Output:**

```
Exponential Smoothing Forecast:
50    456.013060
51    462.701416
52    469.389771
dtype: float64
```

**22. Plot actual and forecasted sales.****Code:**

```
future_dates = pd.date_range(
    start=data.index[-1] + pd.DateOffset(months=1),
    periods=3,
    freq='MS'
)
plt.figure(figsize=(12, 5))
plt.plot(
    data.index,
    data['sales'],
    marker='o',
    label='Actual Sales'
)
plt.plot(
    future_dates,
    es_forecast,
    marker='o',
    linestyle='--',
    label='Forecast'
)
plt.title("Actual and Forecasted Sales")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.legend()
plt.grid(True)
plt.show()
```

**Output:****23. Calculate forecasting error.****Code:**

```
train = data['sales'][:-3]
test = data['sales'][-3:]
es_test_model = ExponentialSmoothing(
    train,
    trend='add',
    seasonal=None
)
```

```
es_test_fit = es_test_model.fit()
es_prediction = es_test_fit.forecast(3)
es_mae = mean_absolute_error(
    test,
    es_prediction
)
print("Exponential Smoothing MAE:", es_mae)
```

**Output:**

```
Exponential Smoothing MAE: 5.166086138528537
```

**24. Apply ARIMA to the same data.****Code:**

```
arima_model = ARIMA(
    data['sales'],
    order=(1, 1, 0)
)
arima_fit = arima_model.fit()
print("ARIMA applied successfully.")
```

**Output:**

```
ARIMA applied successfully.
```

**25. Forecast the next 3 months using ARIMA.****Code:**

```
arima_forecast = arima_fit.forecast(steps=3)
print("ARIMA Forecast:")
print(arima_forecast)
```

**Output:**

```
ARIMA Forecast:
50    455.431732
51    455.450371
52    455.451176
Name: predicted_mean, dtype: float64
```

**26. Compare the ARIMA and Exponential Smoothing forecasts.****Code:**

```
comparison = pd.DataFrame({
    'month': future_dates,
    'Exponential_Smoothing': es_forecast.values,
    'ARIMA': arima_forecast.values
})
```

**29. Conclusion:**

The monthly sales data showed an **overall increasing trend** over time. **Exponential Smoothing** and **ARIMA** models were applied to forecast sales for the next three months. The forecasting errors were calculated using **Mean Absolute Error (MAE)**, and the two models were compared. The model with the **lower MAE** provided the more accurate forecast for this dataset.

```
print("Forecast Comparison:")
print(comparison)
```

**Output:**

```
Forecast Comparison:
   month  Exponential_Smoothing  ARIMA
0 1-04-01          456.013060  455.431732
1 1-05-01          462.701416  455.450371
2 1-06-01          469.389771  455.451176
```

**27. Calculate and compare their forecasting errors.****Code:**

```
arima_test_model = ARIMA(
    train,
    order=(1, 1, 0)
)
arima_test_fit = arima_test_model.fit()
arima_prediction = arima_test_fit.forecast(steps=3)
arima_mae = mean_absolute_error(
    test,
    arima_prediction
)
print("Exponential Smoothing MAE:", es_mae)
print("ARIMA MAE:", arima_mae)
```

**Output:**

```
Exponential Smoothing MAE: 5.166086138528537
ARIMA MAE: 16.642601736440458
```

**28. Identify which model gives the lower forecasting error for the given dataset.****Code:**

```
if es_mae < arima_mae:
    print("Exponential Smoothing gives the lower forecasting error.")
elif arima_mae < es_mae:
    print("ARIMA gives the lower forecasting error.")
else:
    print("Both models have the same forecasting error.")
```

**Output:**

```
Exponential Smoothing gives the lower forecasting error.
```